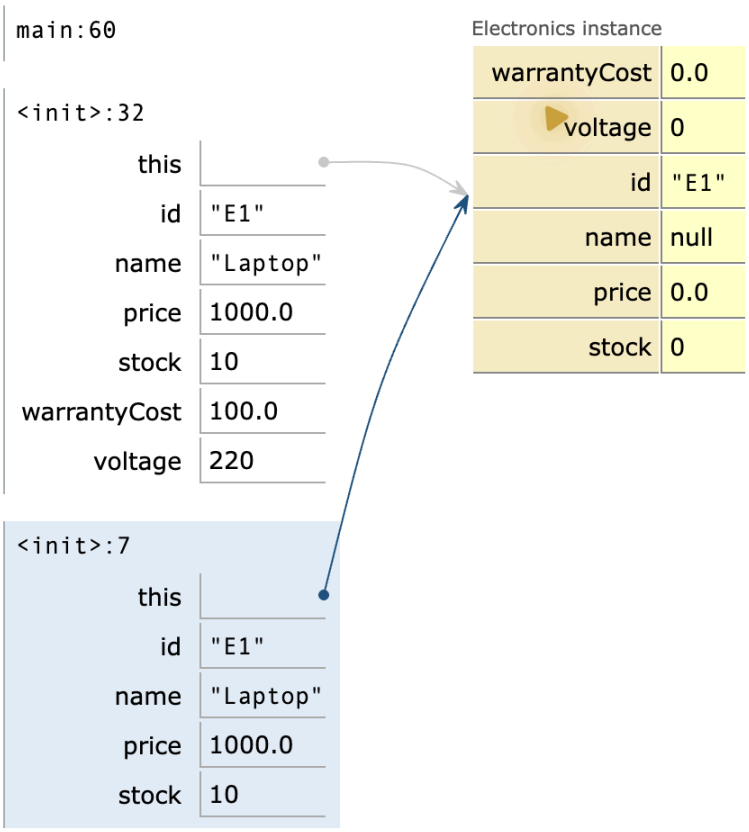
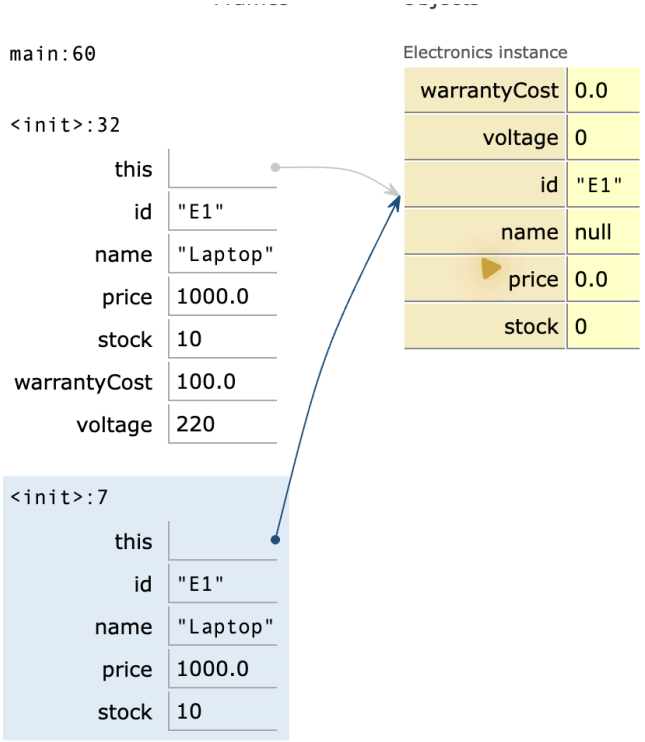
VISUALISING A CODE:



description:(what we have actually understood from the image)

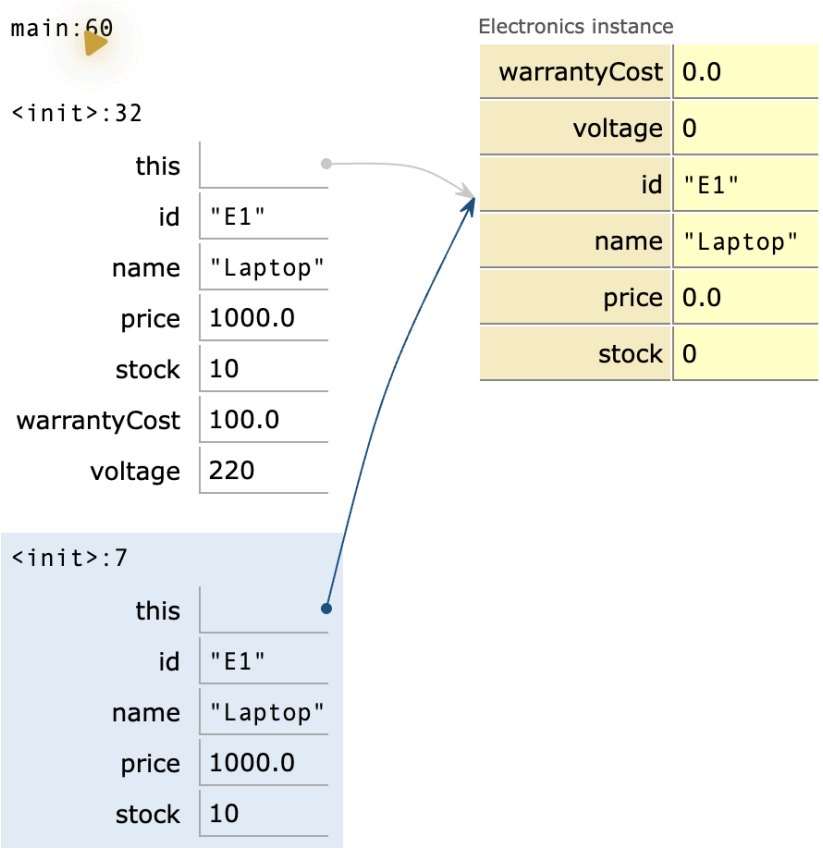
The diagram shows an `Electronics` object being created, where its constructor calls its parent class constructor via `super(...)` to initialize the shared fields (id, name, price, stock). Both constructor frames refer to the same `this` object, which is still holding default values because no assignments have run yet. Once the parent constructor finishes, control returns to the `Electronics` constructor, which then sets its own fields, `warrantyCost` and `voltage`.



description:(what we have actually understood from the image)

The parent constructor is partway through initializing the `Electronics` object, assigning its fields one at a time in order. At this moment `id` has been set to “E1”, while the remaining fields still hold their default values. The child-specific fields, `warrantyCost` and `voltage`, will only be assigned after control returns to the `Electronics` constructor.

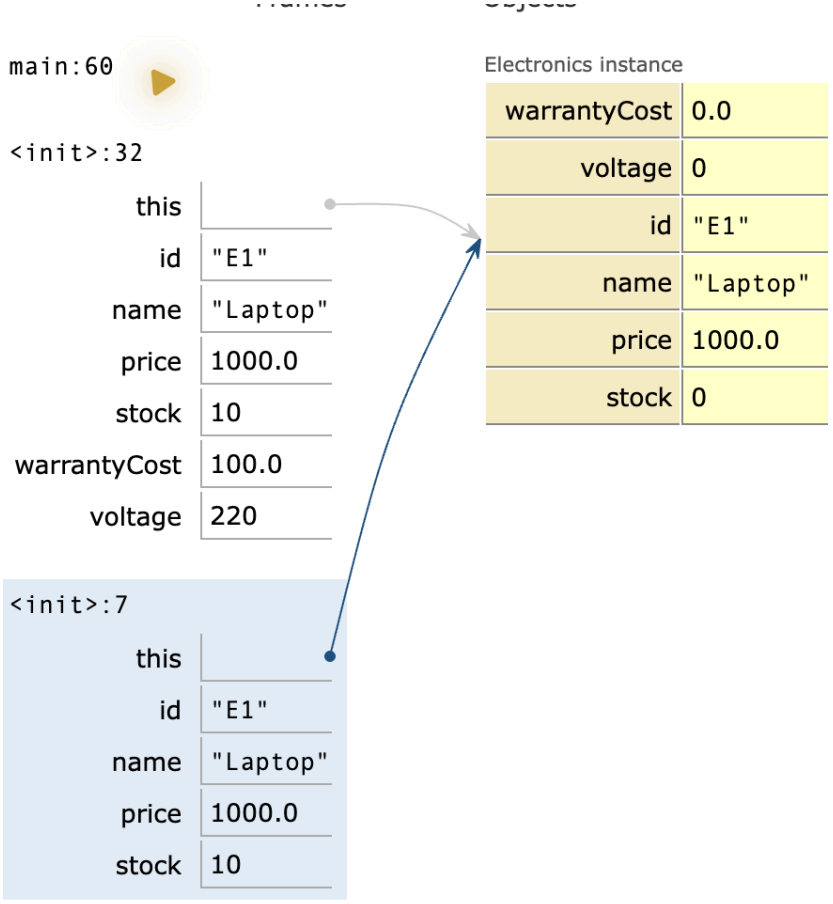
One small note: the arrow position in these diagrams marks the next line to run, so `name` should be the very next to be assigned if the code sets fields in the order id, name, price, stock. The marker appearing at `price` suggests `name`’s assignment may be happening in between steps, so stepping one more time should show it updated.



description:(what we have actually understood from the image)

The parent constructor is continuing to initialize the `Electronics` object one field at a time, and it has now set `id` and `name`. The remaining fields, `price` and `stock`, still hold default values and will be assigned next, followed by the child-specific `warrantyCost` and `voltage` once control returns to the `Electronics` constructor.

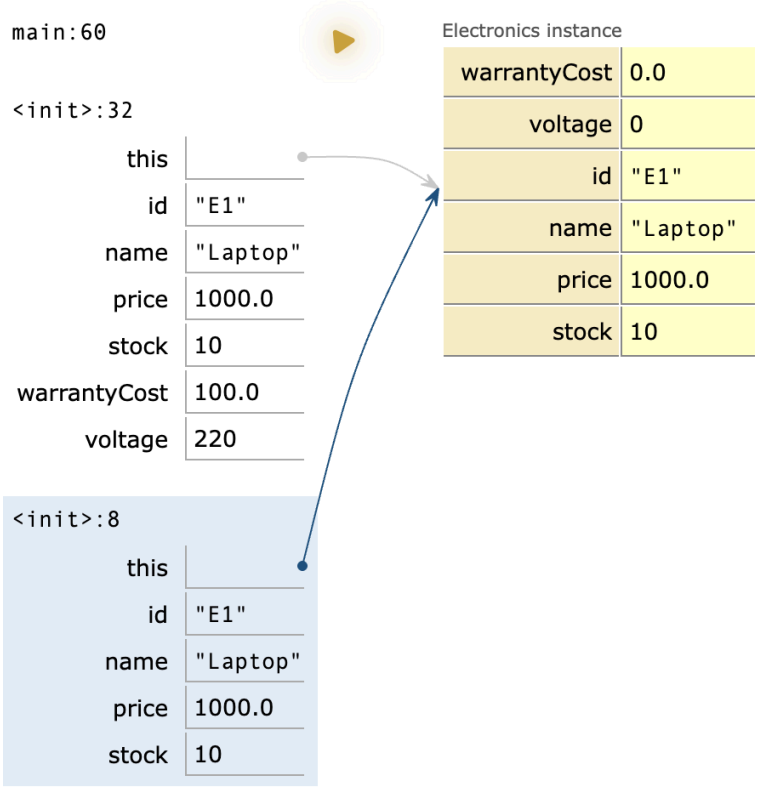
Also, this clears up my earlier note: `name` was indeed assigned right after `id`, so the code is setting fields in the order id, name, price, stock.



description:(what we have actually understood from the image)

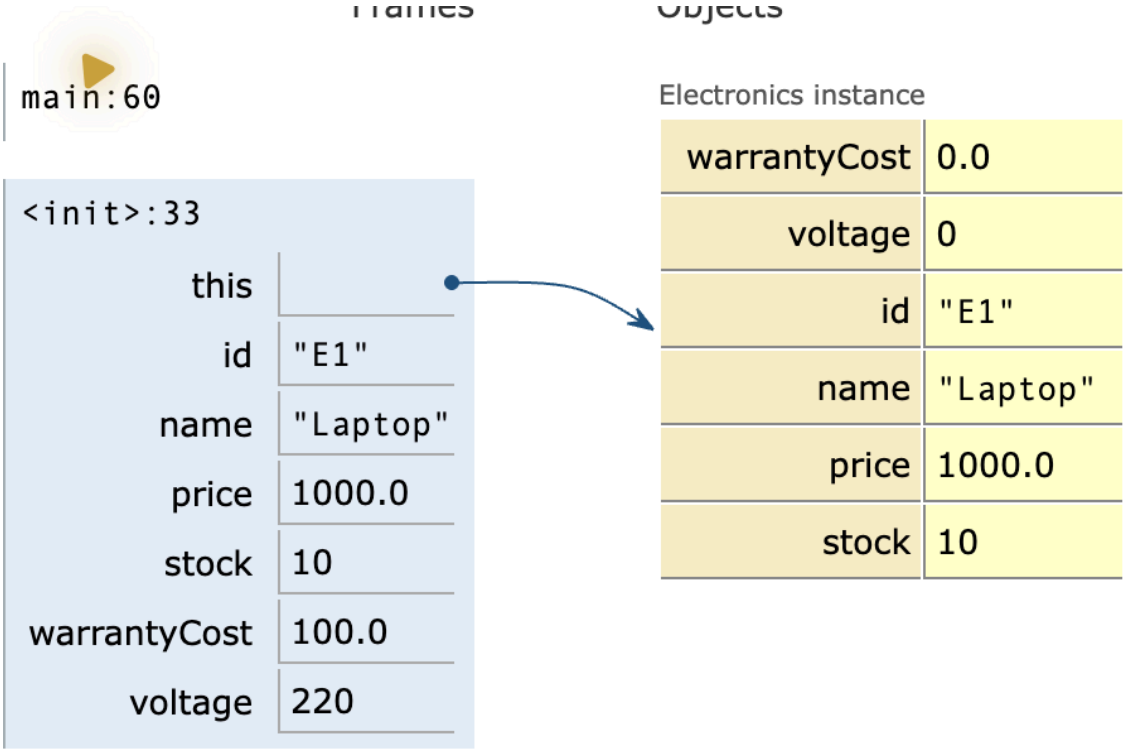
This diagram illustrates **constructor chaining during object instantiation in Java (inheritance)**. The subclass constructor (`<init>:32` of `Electronics`) has invoked the superclass constructor (`<init>:7`) using `super(...)` to initialize inherited fields like `id`, `name`, and `price`. Both active stack frames hold a `this` reference pointing to the same single `Electronics` instance

currently being constructed on the heap.



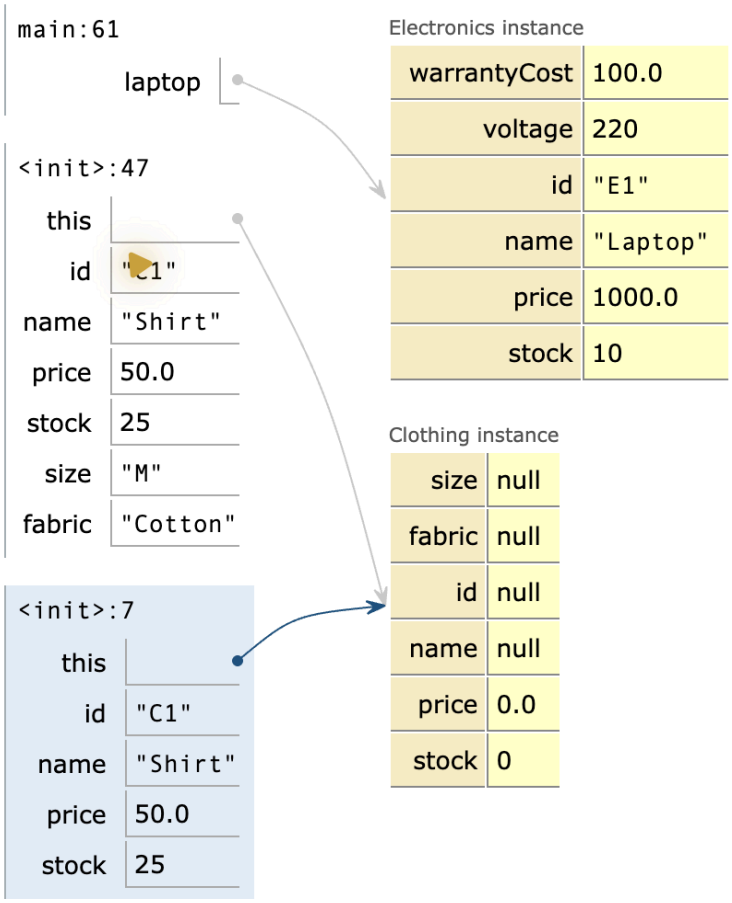
description:(what we have actually understood from the image)

This image shows the **completion of the superclass constructor execution** at line 8 (`<init>:8`). The final parent field, `stock`, has just been assigned its value (`10`) inside the heap's `Electronics instance`, completing the initialization of all inherited properties (`id`, `name`, `price`, `stock`). Control is now about to return back to the subclass constructor (`<init>:32`) to assign its remaining specific fields (`warrantyCost` and `voltage`).



description:(what we have actually understood from the image)

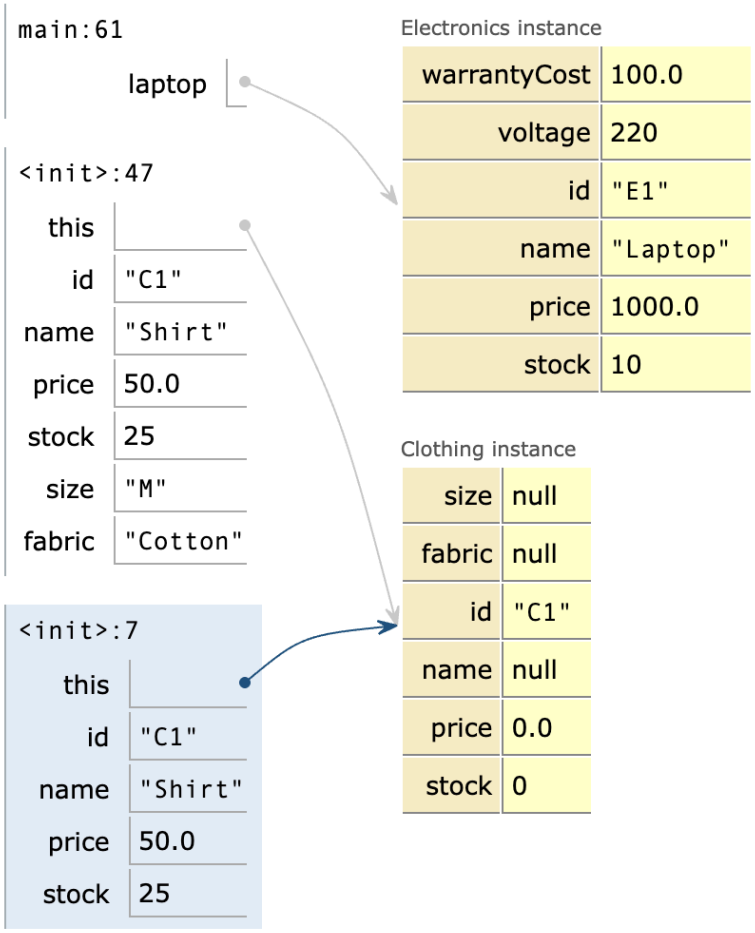
This image shows **control returning to the subclass constructor (`<init>:33`)** after the superclass constructor has finished executing and popped off the call stack. The base properties (`id`, `name`, `price`, `stock`) are fully initialized in the heap object, while the active frame is now ready at line 33 to begin initializing the subclass-specific fields (`warrantyCost` and `voltage`).



description:(what we have actually understood from the image)

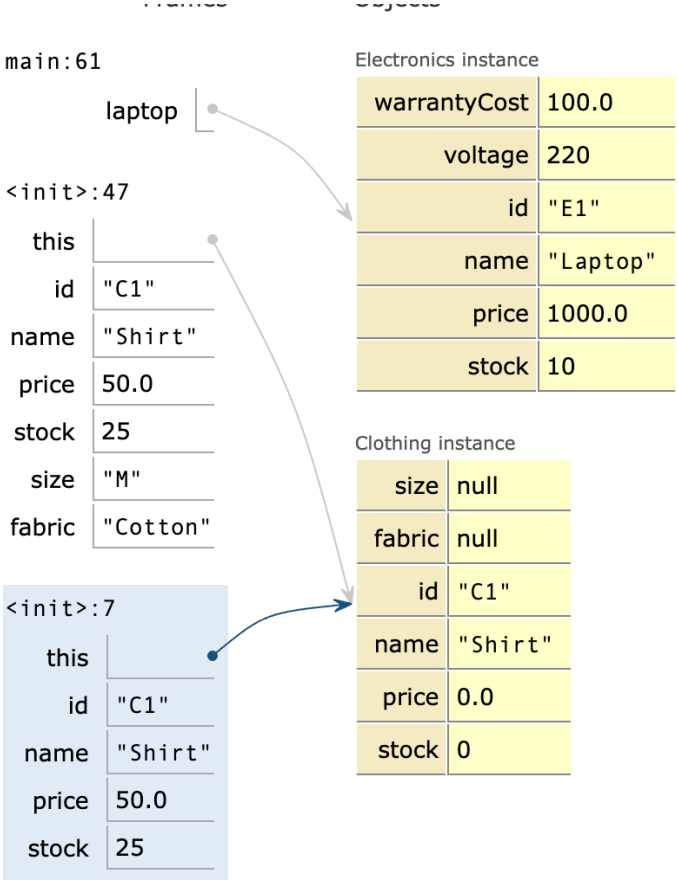
This image illustrates the **instantiation of a second subclass (`Clothing`) within the same inheritance hierarchy**. The previous `Electronics` object is now fully initialized and stored in `main`'s `laptop` variable, while execution at line 61 has started constructing a new `Clothing` object (`"C1"`, `"Shirt"`). The `Clothing` constructor (`<init>:47`) has chained to the shared parent constructor (`<init>:7`), which is currently running to initialize the base

fields for this new instance on the heap.



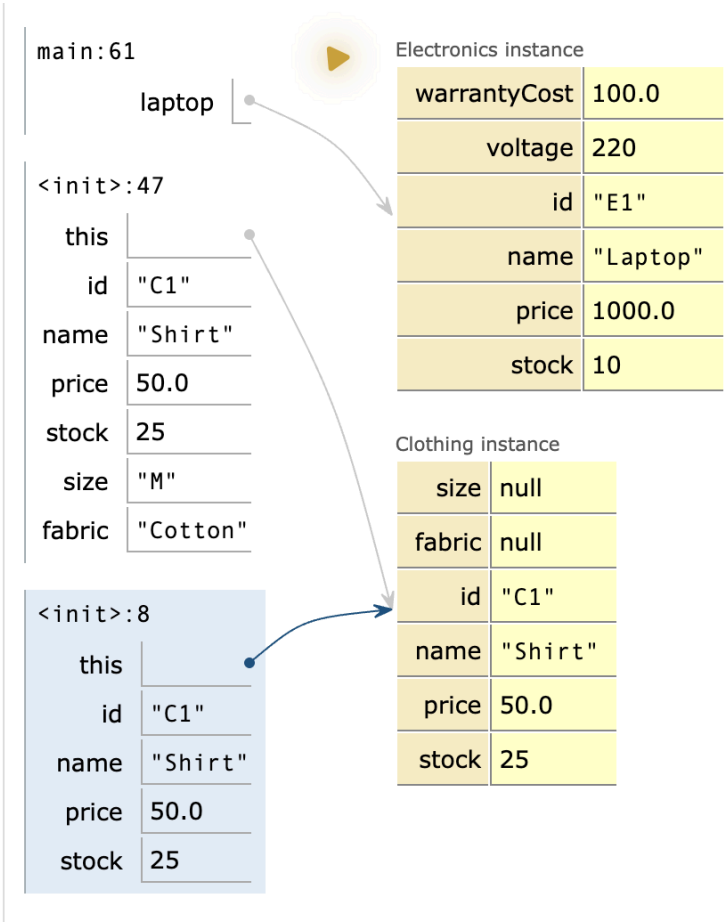
Description:(what we have actually understood from the image)

This image captures the **first assignment executing inside the parent constructor (`<init>:7`)** for the newly allocated `Clothing` object. The statement `this.id = id;` has just completed, updating the `id` attribute on the heap from `null` to `"C1"`. The remaining inherited fields (`name`, `price`, `stock`) are still at their default values, waiting to be populated line-by-line as the parent constructor progresses.



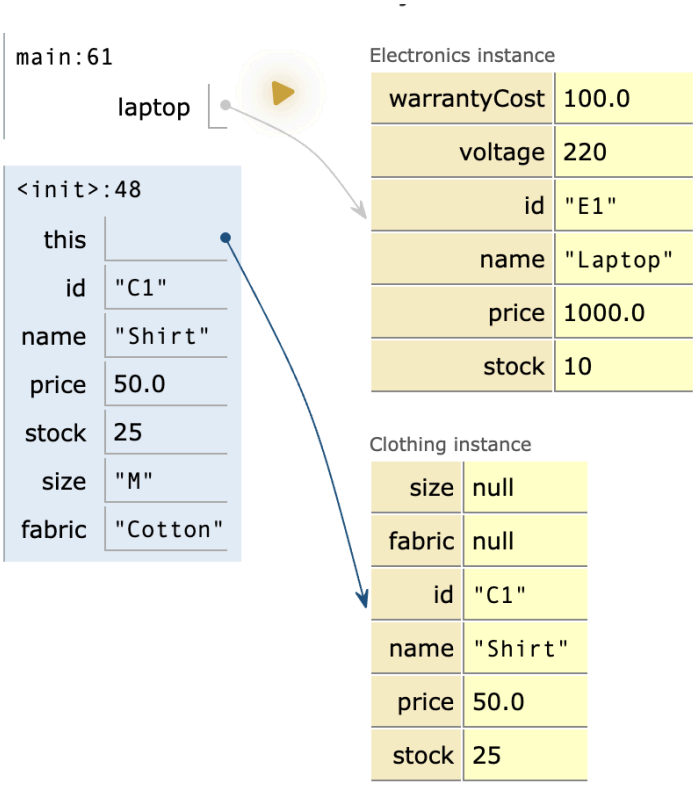
Description:(what we have actually understood from the image)

This image shows the **second assignment executing inside the parent constructor (`<init>:7`)** for the `Clothing` instance. The statement `this.name = name;` has just finished, updating the object's `name` property on the heap from `null` to `"Shirt"`. Both `id` and `name` are now assigned, while the remaining base fields (`price` and `stock`) are next in line to receive their values.



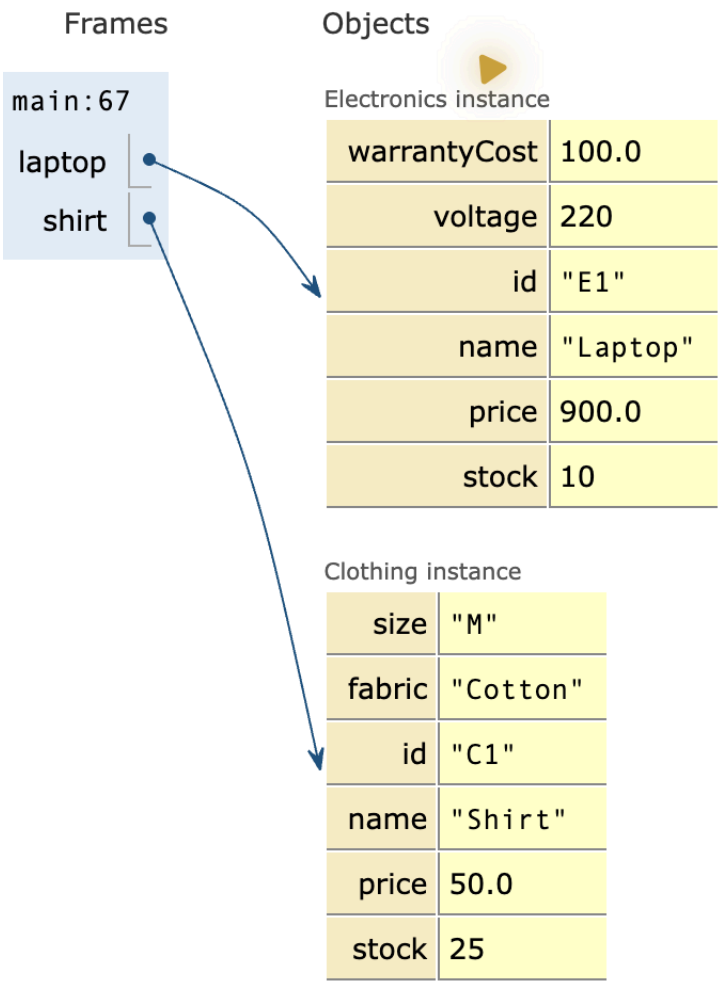
Description:(what we have actually understood from the image)

This image captures the **completion of the parent constructor (`<init>:8`)** for the `Clothing` instance. The statements initializing `price` (`50.0`) and `stock` (`25`) have just finished executing, completing all four inherited base fields on the heap. The parent stack frame is now ready to exit and pop off the stack, returning execution to the `Clothing` constructor (`<init>:47`) to initialize `size` and `fabric`.



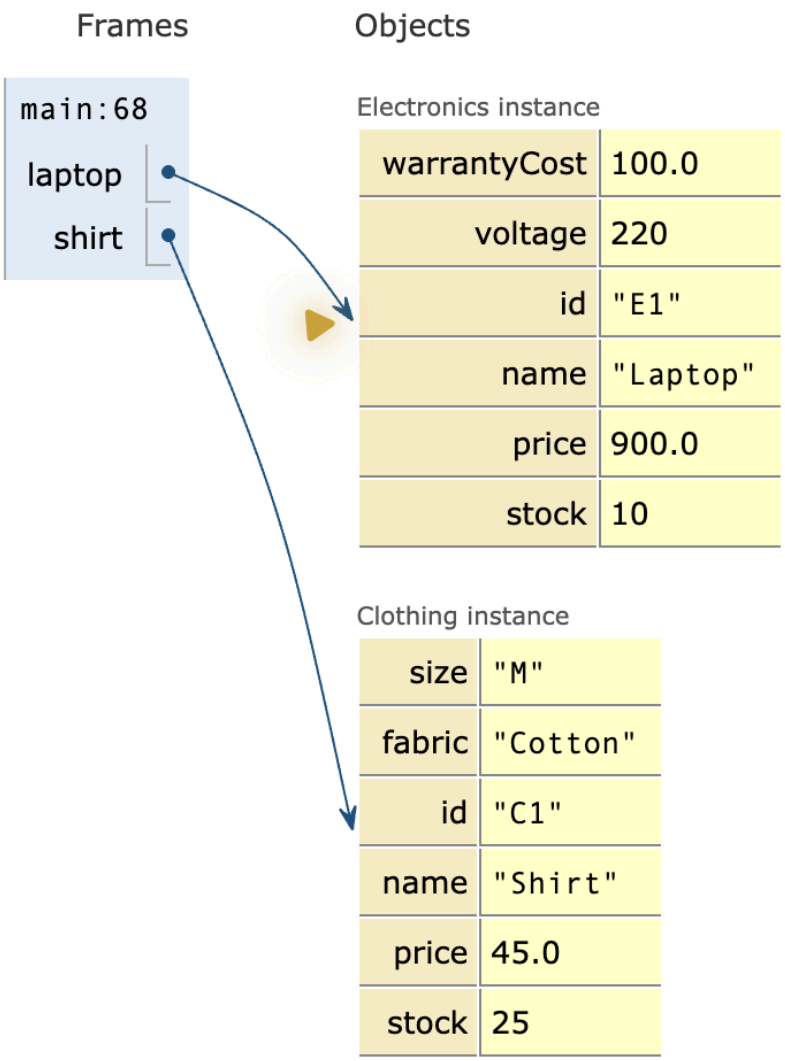
Description:(what we have actually understood from the image)

This image shows **control returning to the `Clothing` subclass constructor (`<init>:48`)** after the parent constructor has finished and popped off the call stack. All inherited fields (`id`, `name`, `price`, `stock`) are now permanently stored in the `Clothing instance` on the heap. The program is currently paused at line 48, ready to assign the subclass-specific fields (`size = "M"` and `fabric = "Cotton"`).



Description:(what we have actually understood from the image)

This image shows **both objects fully instantiated and actively referenced in the `main` method at line 67 (`main:67`)**. Both constructor chains have completed and popped off the stack, leaving `laptop` referencing the `Electronics instance` and `shirt` referencing the `Clothing instance` on the heap. Furthermore, an intervening method call in `main` has modified the `laptop`'s state on the heap, reducing its `price` from `1000.0` to `900.0` (e.g., applying a 10% discount)



Description:(what we have actually understood from the image)

This image captures **execution advancing to line 68 (`main:68`) after applying a discount method to the `shirt` object**. Just as the `laptop`'s price was previously reduced to `900.0`, the `shirt`'s price has now been modified on the heap from `50.0` to `45.0` (a 10% reduction). Both objects now reflect their post-discount states while remaining independently referenced by `laptop` and `shirt` in the `main` stack frame.

MADE BY:
MANVITH A - 1BY25AI132
BHUVAN BHAT - 1BY25AI053